

Assembly Programming Lab Report

System / Environment

Item	Details
Host OS	Windows 10 x64
Linux layer	Ubuntu 24.04 LTS on WSL 2
Tool-chain	NASM 2.16.01 (assembler) · binutils (ld) · GCC 13.2 (for libc linking)
Verified date	23 July 2025

1 Objectives

1. Install NASM and supporting build tools.
2. Write, assemble, link, and run three introductory x86-64 programs:
 - **hello.asm** – prints *Hello, World!*
 - **add.asm** – adds two integers and displays the operands and sum.
 - **star.asm** – prints a three-line star pattern (*, **, ***).
3. Capture build commands, outputs, and any troubleshooting steps.

2 Installation Procedure

1 Enable universe repo & refresh indexes

```
vpuwar@VijayainghLeno:~/asm$ sudo add-apt-repository universe && sudo apt up
date
Adding component(s) 'universe' to all repositories.
Press [ENTER] to continue or Ctrl-c to cancel.
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
138 packages can be upgraded. Run 'apt list --upgradable' to see them.
vpuwar@VijayainghLeno:~/asm$ |
```

2 Install assembler and C tool-chain

```
vpuwar@VijayainghLeno:~/asm$ sudo apt install nasm build-essential
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
nasm is already the newest version (2.16.01-1build1).
build-essential is already the newest version (12.10ubuntu1).
0 upgraded, 0 newly installed, 0 to remove and 138 not upgraded.
```

3 Verification

```
vpuwar@VijayainghLeno:~/asm$ nasm -v
NASM version 2.16.01
vpuwar@VijayainghLeno:~/asm$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

vpuwar@VijayainghLeno:~/asm$ |
```

3 Source Code Listings

1. Create a program that displays Hello World

```
vpuwar@VijayainghLeno:~/asm$ nano hello.asm |
```

After this command this is the source code:

```
GNU nano 7.2 hello.asm
; hello.asm - 64-bit "Hello, World!" for Linux

section .data
    msg    db "Hello, World!", 10      ; string + newline
    len    equ $ - msg                ; length of the string

section .text
    global _start

_start:
    mov     rax, 1                     ; sys_write
    mov     rdi, 1                     ; stdout
    mov     rsi, msg                   ; buffer
    mov     rdx, len                   ; length
    syscall

    mov     rax, 60                    ; sys_exit
    xor     rdi, rdi                   ; status 0
    syscall
```

2. Create a program that adds two numbers and displays both

Creating a add.asm file

```
vpuwar@VijayainghLeno:~/asm$ nano add.asm
```

Here is the source code:

```
GNU nano 7.2 add.asm
;-----
; add.asm - prints "7 + 5 = 12" using libc printf
;-----
default rel
section .data
    fmt    db    "%d + %d = %d", 10, 0      ; "%d + %d = %d\n"
    num1   dd    7
    num2   dd    5

section .text
    global main
    extern printf

main:
    ; load arguments for printf(format, n1, n2, sum)
    mov     rax, [rel num1]      ; rax = num1
    mov     esi, eax             ; 2nd arg (rsi) = num1
    mov     eax, [rel num2]
    mov     edx, eax             ; 3rd arg (rdx) = num2
    mov     eax, [rel num1]
    add     eax, [rel num2]      ; eax = num1 + num2
    mov     ecx, eax             ; 4th arg (rcx) = sum
    lea     rdi, [rel fmt]       ; 1st arg (rdi) = format string
    xor     eax, eax             ; SysV ABI: rax = 0 for variadic funcs
    call    printf

    ; return 0
    xor     eax, eax
    ret
```

3. Create a program that displays the following shape on the screen

Creating the star.asm file

```
vpuwar@VijayainghLeno:~/asm$ nano star.asm
```

Source Code:

```

GNU nano 7.2                                star.asm
; star.asm - prints:
; *
; **
; ***

default rel                                ; RIP-relative addressing

section .data
    stars db "*",10                        ; "\n"
           db "**",10                       ; "**\n"
           db "***",10                     ; "***\n"
    stars_len equ $ - stars                ; total length = 1+1 + 2+1 + 3+1 = 9

section .text
    global _start

; ----- pure syscalls: no printf, no GCC hardening flags -----
_start:
    mov     rax, 1                          ; sys_write
    mov     rdi, 1                          ; fd = stdout
    lea     rsi, [rel stars]; buffer
    mov     rdx, stars_len ; length
    syscall

    mov     rax, 60                         ; sys_exit
    xor     rdi, rdi                        ; status = 0
    syscall

section .note.GNU-stack progbits alloc noexec

```

4 Build & Execution Commands

Program	Assemble	Link	Run	Expected Output
hello	nasm -f elf64 hello.asm -o hello.o	ld hello.o -o hello	./hello	Hello, World!
add	nasm -f elf64 add.asm -o add.o	gcc -no-pie add.o -o add	./add	7 + 5 = 12
star	nasm -f elf64 star.asm -o star.o	ld star.o -o star	./star	*****

5 Troubleshooting Log

Symptom	Cause	Resolution
E: Unable to locate package nasm	<i>universe</i> repo disabled	add-apt-repository universe then apt update

relocation ... can not be used when making a PIE object while linking add.o	GCC links PIE by default; object isn't PIC	Added -no-pie flag
Garbage characters after first star line	Missing NUL terminator or incorrect mov vs lea	Re-wrote program to use a single write buffer or ensured each string ends with , 0

6 Verification

All programs execute with correct output and zero exit status.

```
vpuwar@VijayainghLeno:~/asm$ nasm -f elf64 hello.asm -o hello.o
vpuwar@VijayainghLeno:~/asm$ ld hello.o -o hello
vpuwar@VijayainghLeno:~/asm$ ./hello
Hello, World!
vpuwar@VijayainghLeno:~/asm$ nasm -f elf64 add.asm -o add.o
vpuwar@VijayainghLeno:~/asm$ gcc -no-pie add.o -o add
/usr/bin/ld: warning: add.o: missing .note.GNU-stack section implies executable stack
/usr/bin/ld: NOTE: This behaviour is deprecated and will be removed in a future version of the linker
vpuwar@VijayainghLeno:~/asm$ ./add
7 + 5 = 12
vpuwar@VijayainghLeno:~/asm$ nasm -f elf64 star.asm -o star.o
vpuwar@VijayainghLeno:~/asm$ ld star.o -o star
vpuwar@VijayainghLeno:~/asm$ ./star
*
**
***
vpuwar@VijayainghLeno:~/asm$ |
```

7 Conclusions & Next Steps

- The environment is now fully set up for x86-64 assembly on Ubuntu/WSL.
- Demonstrated both **pure-syscall** style (hello, star) and **libc-based** style (printf).
- Future exercises could include loops, command-line argument parsing, and interaction with C functions.

